

24BCF10059 - Lakshdeep

Experiment 1

Aim :

To study and implement advanced SQL queries using conditional logic, aggregate functions, joins, subqueries, self-joins, and date operations to solve real-world database problems.

Question 1

Write an SQL solution to calculate the total number of bank accounts belonging to each salary category. The classification categories are:

1. **"Low Salary"**: All monthly salaries strictly less than \$20,000.
2. **"Average Salary"**: All monthly salaries in the inclusive range [\$20,000, \$50,000].
3. **"High Salary"**: All monthly salaries strictly greater than \$50,000. The final result table must report all three categories, even if a category contains zero accounts.

Solution :

```
SELECT 'LOW SALARY' AS CATEGORY, COUNT (ACCOUNT_ID) AS ACCOUNT_COUNT  
FROM ACCOUNTS
```

```
WHERE INCOME < 20000
```

```
UNION ALL
```

```
SELECT 'AVERAGE SALARY' AS CATEGORY, COUNT (ACCOUNT ID) AS ACCOUNT  
COUNT FROM ACCOUNTS
```

```
WHERE INCOME >= 20000 AND INCOME <= 50000
```

```
UNION ALL
```

```
SELECT 'HIGH SALARY' AS CATEGORY, COUNT(ACCOUNT_ID) AS ACCOUNT_COUNT  
FROM ACCOUNTS WHERE INCOME >50000;
```

Question 2

Design an Employee Management System database architecture by performing the following operations: Create Departments, Employees and Salaries tables. Insert sample records. Display employee details using JOIN. Increase HR employee salary by 10%. Delete salary records below 30000. Display employees earning more than the company average salary.

Solution :

```
SELECT E.*,D.*,S.*
```

```
FROM Employees as E
```

```
INNER JOIN
```

```
Departments as D
```

```
ON E.dept_id = D.dept_id
```

```
INNER JOIN
```

```
Salaries as S
```

```
ON E.emp_id = S.emp_id;
```

```
UPDATE Salaries
```

```
SET salary = salary - 0.1*salary
```

```
WHERE emp_id IN(
```

```
    SELECT emp_id
```

```
    FROM Employees as E
```

```
    INNER JOIN
```

```
    Departments as D
```

```
    ON E.dept_id = D.dept_id
```

```
    WHERE D.dept_id = 10
```

```
);
```

```
DELETE FROM Salaries WHERE salary <30000;
```

```
SELECT E.emp_id
FROM Employees AS E
INNER JOIN Salaries AS S
ON E.emp_id = S.emp_id
WHERE S.salary > (SELECT AVG(salary) FROM Salaries);
```

Question 3

Generate every possible pizza consisting of exactly three different toppings. Display the topping names separated by commas along with the total ingredient cost. Sort the result by highest cost and alphabetically for ties.

Solution :

```
SELECT CONCAT(T1.topping_name,',',T2.topping_name,',',T3.topping_name) AS PIZZA,  
           (T1.ingredient_cost + T2.ingredient_cost + T3.ingredient_cost) AS COST  
FROM pizza_toppings AS T1  
INNER JOIN  
pizza_toppings AS T2  
ON T1.topping_name < T2.topping_name  
INNER JOIN  
pizza_toppings AS T3  
ON T2.topping_name < T3.topping_name;
```

Learning Outcomes :

After completing this experiment, the following learning outcomes were achieved.

- Understood advanced SQL query formulation. Applied conditional logic using CASE statements.
- Implemented aggregate functions and GROUP BY operations.
- Performed table creation, insertion, update and deletion operations.
- Applied INNER JOIN and SELF JOIN for relational data retrieval.
- Used subqueries to perform complex filtering and updates.
- Generated unique combinations using self joins.
- Performed date arithmetic for transaction analysis.
- Improved practical knowledge of PostgreSQL and relational database management.